UNIVERSIDADE FEDERAL DE CIÊNCIA E INOVAÇÃO DE MINAS GERAIS
Engenharia de Computação - Linguagens de Programação
Projeto: funcionalidades de uma linguagem de programação

Ruby

Aluno: Iago Santana | Repositório: github.com/santana-iago/ruby-studycase

Introdução

Meu primeiro contato com Ruby foi há alguns anos, por meio do blog AkitaOnRails, do Fabio Akita. Na época eu não tinha a mesma bagagem de hoje, então resolvi reler algumas publicações e ver alguns vídeos dele novamente, e foi assim que eu desenvolvi esse caso de estudo e vou mencionar o Akita e o blog dele diversas vezes durante esse relatório.

O Akita é provavelmente a forma pela qual eu mais consumo conteúdo técnico de computação fora das aulas. Hoje em dia, diferentemente da época do blog a respeito de Ruby, o tema central é inteligência artificial prática no geral, LLMs, agentes, benchmarks e experimentos que ele publica. Professor Andrei: como você comentou em sala que ainda não conhecia o trabalho dele, fica a indicação. Mesmo quando discordo de alguma conclusão, sempre encontro ali um material de muita qualidade para estudar [1–7].

O que mais me interessa em Ruby não é a promessa de escrever menos linhas. É a liberdade da linguagem e o fato de ela recompensar quem conhece seus mecanismos de verdade. Um programador mais experiente consegue montar APIs muito expressivas com blocos, closures, módulos e metaprogramação; mas também sabe quando usar esses recursos só tornaria o projeto difícil de manter. Essa diferença entre conhecer a ferramenta e saber usá-la bem é o centro do relatório.

Ruby costuma ser tratado quase como sinônimo de Ruby on Rails, e não é por acaso. O framework teve um papel enorme na divulgação da linguagem e usa muita coisa que existe no Ruby puro para criar formas mais expressivas e intuitivas de escrever código. Mas são duas camadas diferentes. Por isso escolhi oito exemplos exclusivamente de Ruby nativo; depois mostro o que Rails acrescenta ao mesmo tipo de código. Em cada funcionalidade também faço uma comparação com outra linguagem para fins de comparação e associação.

Os oito exemplos em Ruby puro

PS: usei o Ruby 3.3.8 nos testes

1. Tudo é objeto (inclusive o operador de soma)

Código

```
numero = 7
puts numero.class
puts numero.+(5)
puts numero.method(:+).call(5)
puts true.class
puts nil.class
puts String.class
puts :ruby.class
puts numero.respond_to?(:times)
```

Explicação

Esse é o primeiro detalhe de Ruby que eu considero realmente diferente de várias linguagens que já usei. O número 7 não é simplesmente um valor solto: ele é uma instância de Integer, e o + é um método que pode ser chamado como numero.+(5). A chamada numero.method(:+) ainda devolve um objeto Method, que posso executar com call. A própria classe String é um objeto da classe Class; true, nil e :ruby também têm suas classes. O respond_to? consulta quais mensagens um objeto aceita.

Justificativa

Não acho isso interessante só porque a sintaxe é diferente. A ideia de enviar mensagens para objetos aparece com muita consistência, inclusive em operações básicas. Para mim, entender isso ajuda bastante antes de tentar brincar com reflexão e metaprogramação. O primeiro Micro-Tutorial do Akita foi uma boa referência para enxergar esse modelo. [1] [9]

Comparação com Java

Java faz uma distinção explícita entre um int primitivo e o wrapper Integer. Em Ruby, o inteiro já responde a métodos no próprio modelo da linguagem. Isso não quer dizer que as representações internas sejam iguais nem que um seja automaticamente mais rápido. Na comparação Java do repositório, mostro justamente primitivo, wrapper e a consulta à classe.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
int primitivo = 7;
Integer objeto = Integer.valueOf(primitivo);
System.out.println(objeto.getClass().getSimpleName());
```

Figura 2 - saída obtida ao executar o exemplo 2 com Ruby 3.3.8.

```
$ ruby exemplos/ruby/01_tudo_e_objeto.rb
Integer
12
12
TrueClass
NilClass
Class
Symbol
true
```

Código completo da comparação: comparacoes/java/Main.java (no repositório).

2. Um método que só existe em um objeto

Código

```
perfil = Object.new
def perfil.apresentar = "Sou um objeto com metodo proprio"
outro = Object.new
puts perfil.apresentar
puts perfil.singleton_methods.inspect
puts outro.respond_to?(:apresentar)
puts perfil.singleton_class.superclass
```

Explicação

Nesse trecho, eu crio dois objetos da mesma classe, mas adiciono apresentar somente ao primeiro. O método foi definido no objeto perfil, não em Object de forma global. singleton_methods lista essa diferença; singleton_class.superclass ajuda a visualizar a classe anônima envolvida. É um recurso nativo do Ruby, não algo que o Rails coloca por cima.

Justificativa

Aqui começa uma das partes mais interessantes do Ruby, a liberdade é enorme, mas também vem com um custo: alterar comportamentos de objetos isolados sem critério pode deixar o código bem difícil de rastrear. Eu usaria com cuidado, sobretudo em testes e pequenas adaptações.

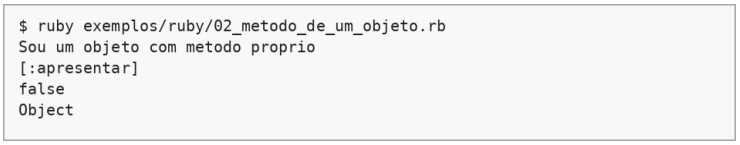
Comparação com JavaScript

Em JavaScript, posso atribuir uma função como propriedade de um objeto e deixar outro sem essa propriedade. O efeito observado é parecido, mas os mecanismos são diferentes: propriedades com funções em JavaScript, singleton class e busca de métodos em Ruby. É uma comparação útil justamente porque a aparência parecida pode enganar.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
const perfil = {};
perfil.apresentar = () => "Sou um objeto com metodo proprio";
console.log(Object.hasOwn(perfil, "apresentar"));
```

Figura 2 - saída obtida ao executar o exemplo 2 com Ruby 3.3.8.



Código completo da comparação: comparacoes/javascript/02_metodo_por_objeto.mjs (no repositório).

3. Closures: funções que guardam o contexto

Código

```
def criar_desconto(porcentagem)
  ->(preco) { (preco * (1 - porcentagem / 100.0)).round(2) }
end
cupom = criar_desconto(25)
precos = [50, 100, 140]
puts cupom.class
puts precos.map(&cupom).inspect
puts cupom.call(80)
puts criar_desconto(10).call(80)
```

Explicação

criar_desconto devolve uma lambda (um Proc) que continua tendo acesso à variável porcentagem depois que a execução do método terminou. Isso é uma closure: código acompanhado do ambiente léxico que ele capturou. Depois, map(&cupom) converte o Proc em bloco e aplica a

mesma regra a cada preço. O resultado não é só um número: é comportamento que posso passar de um lugar para outro.

Justificativa

Achei esse um dos recursos mais naturais do Ruby. A forma como blocos se encaixam nos métodos permite escrever regras reutilizáveis sem criar uma classe nova para tudo. É também o tipo de funcionalidade que faz muita API Ruby parecer uma pequena linguagem própria. O artigo Anatomia de Ruby Blocks/Closures, do Akita, explica bastante coisa por trás disso. [2] [4]

Comparação com JavaScript

JavaScript também tem closures de primeira classe e consegue fazer a mesma regra de desconto sem dificuldade. A diferença está mais na maneira de expressar e passar o comportamento: no JavaScript, callback em map; no Ruby, Proc e a convenção do bloco com &. Não é uma invenção exclusiva do Ruby, mas é muito bem integrada à linguagem.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
function criarDesconto(p) {
  return preco => +(preco * (1 - p / 100)).toFixed(2);
}
```

Figura 3 - saída obtida ao executar o exemplo 3 com Ruby 3.3.8.

```
$ ruby exemplos/ruby/03_closure_e_proc.rb
Proc
[37.5, 75.0, 105.0]
60.0
72.0
```

Código completo da comparação: comparacoes/javascript/03_closure.mjs (no repositório).

4. Enumeradores lazy e uma sequência sem fim

Código

```
potencias = Enumerator.produce(1) { |n| n * 2 }
filtro = potencias.lazy.select { |n| n >= 100 }
puts filtro.first(6).inspect
puts potencias.class
puts filtro.class
puts filtro.take(3).force.inspect
```

Explicação

Enumerator.produce cria uma sequência de potências de dois que, em princípio, não termina. O detalhe que deixa esse código seguro é a avaliação sob demanda. Com lazy, select não tenta percorrer tudo imediatamente. Quando peço first(6), o Ruby só percorre o suficiente para descobrir os seis primeiros valores que atendem à condição. A segunda chamada com take(3).force deixa o comportamento ainda mais evidente.

Justificativa

É uma boa amostra de como compor operações simples pode resolver um problema real. Em processamento de grandes sequências, não materializar tudo de uma vez pode ser uma vantagem importante de memória. Mas lazy não é uma otimização mágica: em muitos casos pequenos, usar operações diretas é mais simples e suficiente. [15]

Comparação com Python

Python oferece generators e itertools, capazes de produzir uma sequência infinita e consumir só uma parte dela. Aqui as duas linguagens têm boas soluções. O Ruby chama atenção pela forma de encadear objetos Enumerator e Enumerator::Lazy, enquanto Python costuma expressar o mesmo fluxo com expressões geradoras e islice.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
from itertools import count, islice
potencias = (2 ** n for n in count())
print(list(islice((x for x in potencias if x >= 100), 6)))
```

Figura 4 - saída obtida ao executar o exemplo 4 com Ruby 3.3.8.

```
$ ruby exemplos/ruby/04_enumerator_lazy.rb
[128, 256, 512, 1024, 2048, 4096]
Enumerator
Enumerator::Lazy
[128, 256, 512]
```

Código completo da comparação: comparacoes/python/04_lazy.py (no repositório).

5. Metaprogramação em Ruby: criando métodos em tempo de execução

Código

```
class Config
  def self.opcao(nome, padrao:)
    define_method(nome) { @valores.fetch(nome, padrao) }
  end
  def initialize(**valores) = @valores = valores
  opcao :cache, padrao: true
  opcao :idioma, padrao: "pt-BR"
end
puts Config.new(cache: false).cache
puts Config.new.idioma
```

Explicação

O método de classe opcao recebe o nome de uma configuração e cria dinamicamente um novo método de instância com define_method. Quando o corpo da classe executa opcao :cache, padrao: true, aquilo é só uma chamada de método Ruby que adiciona comportamento à classe. A instrução @valores.fetch busca o valor definido pelo usuário ou aplica o padrão. A sintaxe parece uma declaração especial, mas não existe nenhuma palavra reservada opcao no interpretador.

Justificativa

Esse foi provavelmente o exemplo que eu mais gostei de estudar. É aqui que entendo por que um programador que conhece Ruby de verdade consegue criar APIs tão expressivas, mas também por que essa liberdade exige responsabilidade. É muito fácil fazer uma DSL elegante virar algo impossível de depurar se for usada sem necessidade. O Akita aborda métodos dinâmicos no terceiro Micro-Tutorial. [3] [5]

Comparação com Python / Ruby on Rails

Python consegue criar propriedades dinamicamente com setattr e property. O resultado pode ser semelhante, embora a forma de declarar seja diferente. E aqui aparece a ponte com Rails: chamadas como validates :cliente, presence: true parecem novas palavras da linguagem, mas são métodos fornecidos pelo ActiveRecord, escritos graças aos recursos do Ruby. O define_method do exemplo é Ruby puro; validates é Rails.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
def opcao(nome, padrao):
    setattr(Config, nome, property(
        lambda self: self.valores.get(nome, padrao)))
```

Figura 5 - saída obtida ao executar o exemplo 5 com Ruby 3.3.8.

```
$ ruby exemplos/ruby/05_define_method_dsl.rb
false
pt-BR
```

Código completo da comparação: comparacoes/python/05_metaprogramacao.py (no repositório).

6. prepend, super e a cadeia de métodos

Código

```
Pedido = Class.new { def finalizar = "pedido confirmado" }
module Auditoria
  def finalizar
    puts "[log] iniciando finalizacao"
    super
  end
end
Pedido.prepend(Auditoria)
puts Pedido.new.finalizar
```

Explicação

O módulo Auditoria tem seu próprio método finalizar, que registra uma mensagem e chama super. Ao executar Pedido.prepend(Auditoria), o Ruby coloca esse módulo antes de Pedido na ordem de busca de métodos. Assim, a chamada primeiro passa pela auditoria e só depois chega ao método original. No início, Class.new também mostra que classes podem ser construídas como objetos.

Justificativa

Isso tem cara de uma funcionalidade que pode ser muito útil em instrumentação, auditoria e adaptações de comportamento sem reescrever o método original. Mas um sistema com muitos módulos usando prepend exige que a equipe entenda muito bem a cadeia de lookup. O ganho de flexibilidade só vale quando a solução continua legível. [9] [10]

Comparação com Python

Python permite um desenho semelhante com herança múltipla, mixins e sua MRO (method resolution order). Não é o mesmo mecanismo: no Python, a ordem é derivada da hierarquia de classes; em Ruby, prepend altera a cadeia de busca de métodos da própria classe. Nos dois casos, entender o que super fará exige olhar a resolução de métodos.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
class PedidoAuditado(Auditoria, Pedido):
    pass
print(PedidoAuditado().finalizar())
```

Figura 6 - saída obtida ao executar o exemplo 6 com Ruby 3.3.8.

```
$ ruby exemplos/ruby/06_prepend_e_super.rb
[log] iniciando finalizacao
pedido confirmado
```

Código completo da comparação: comparacoes/python/06_mro.py (no repositório).

7. Refinements: mudar um comportamento sem mexer no mundo inteiro

Código

```
module Slug
  refine String do
    def slug = downcase.strip.gsub(/\s+/, "-")
  end
end
puts "Ruby on Rails".respond_to?(:slug)
using Slug
puts " Ruby on Rails  ".slug
puts String.instance_methods(false).include?(:slug)
```

Explicação

O módulo Slug define um refinement para String, acrescentando o método slug que normaliza um texto. Antes do using, o objeto não responde a slug. Depois de ativar o refinement, a chamada funciona no escopo léxico em que ele está ativo. A consulta a String.instance_methods(false) mostra que não acrescentei esse método diretamente à tabela global de métodos de String.

Justificativa

Conhecer Ruby também significa entender as armadilhas do chamado monkey patching: abrir uma classe central e afetar o restante do programa. Refinements são uma maneira de reduzir esse efeito colateral. Não são isolamento absoluto e têm regras de escopo próprias, então precisam de cuidado; justamente por isso achei um bom exemplo para o trabalho. [11]

Comparação com C#

C# tem extension methods, que parecem métodos acrescentados a uma classe. A semelhança é a ergonomia. A diferença é importante: um extension method é resolvido estaticamente pelo compilador e não injeta comportamento real em string; o refinement participa da resolução dinâmica de métodos no escopo Ruby. O trecho em C# no repositório é ilustrativo, sem teste .NET neste trabalho.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
public static string Slug(this string valor) =>
    string.Join("-", valor.Trim().ToLowerInvariant()
        .Split(' ', StringSplitOptions.RemoveEmptyEntries));
```

Figura 7 - saída obtida ao executar o exemplo 7 com Ruby 3.3.8.



Código completo da comparação: comparacoes/csharp/07_refinements_vs_extensions.cs (no repositório).

8. Pattern matching com Hash, intervalo e captura

Código

```
evento = { tipo: :pedido, dados: { id: 42, valor: 180 } }
case evento
in { tipo: :pedido, dados: { id:, valor: (100.. => valor) } }
  puts "Pedido #{id}: frete gratis (R$ #{valor})"
in { tipo: :pedido, dados: { id: } }
  puts "Pedido #{id}: frete normal"
else
  puts "Evento nao reconhecido"
end
```

Explicação

O evento representa um pedido armazenado em um Hash aninhado. Com case ... in, Ruby verifica a forma dos dados e extrai id e valor para usar no corpo do caso. A expressão (100.. => valor) junta duas ideias: exige um valor dentro de um intervalo sem limite superior e, se ele corresponder, vincula esse valor à variável. Se esse padrão não se aplicar, a próxima alternativa ainda pode aceitar o pedido.

Justificativa

Em código que recebe eventos, mensagens ou dados estruturados, acho melhor enxergar a forma esperada da entrada de maneira explícita do que esconder tudo em uma coleção de ifs e verificações de chave. Esse é um recurso do Ruby moderno, sem ligação com Rails. Ele precisa ser usado com parcimônia para não virar um padrão ilegível. [16]

Comparação com Python

O match/case do Python também desestrutura dicionários. Para impor o limite mínimo de 100 no exemplo Python, escrevi uma condição de guarda, if valor >= 100. As sintaxes têm semelhanças, mas as regras de correspondência e a forma de validar valores não são idênticas.

Trecho ilustrativo da comparação (fora dos oito exemplos avaliados):

```
match evento:
    case {"tipo": "pedido", "dados": {"id": ident, "valor": v}} if v >= 100:
        print(f"Pedido {ident}: frete gratis (R$ {v})")
```

Figura 8 - saída obtida ao executar o exemplo 8 com Ruby 3.3.8.



Código completo da comparação: comparacoes/python/08_pattern_matching.py (no repositório).

E onde entra Ruby on Rails?

No caso do Rails eu entendo por que Ruby ficou tão associado à web. O framework não é só um conjunto de funções para responder requisições HTTP: ele entrega convenções para organizar models, controllers, rotas, persistência, migrations, validações e testes. Em muitos sistemas web convencionais isso reduz trabalho repetitivo e deixa a equipe gastar mais tempo nas regras de negócio. O Ruby fornece a expressividade; o Rails fornece a estrutura e uma coleção de bibliotecas.

Por exemplo, no quinto código eu criei a minha própria declaração opcao :cache, padrao: true com define_method. Em um model do Rails dá para escrever validates :cliente, presence: true. Visualmente parecem comandos de uma linguagem especial, mas são chamadas de métodos Ruby. A diferença é que define_method está no Ruby e validates é uma API implementada pelo ActiveRecord. Da mesma maneira, um Hash com dados de um pedido e um pattern matching são Ruby puro; ao criar Pedido < ApplicationRecord e configurar resources :pedidos, já estou usando serviços do Rails.

Uma vantagem do Rails é começar rápido quando a aplicação exige banco, páginas ou API, validação e tarefas web comuns. Mas eu não o escolheria para toda classe de problema. Treinamento de modelos, análise científica pesada e ecossistemas de IA naturalmente favorecem Python; sistemas de baixo nível podem pedir Rust ou C++; e sistemas de mensagens em grande escala podem tirar vantagem de outras runtimes. A escolha sensata depende do uso. No artigo de 11/10/2026, o Akita compara várias linguagens e também Ruby puro com Rails num teste envolvendo filas; o próprio texto mostra a importância de entender o gargalo antes de anunciar qual tecnologia é melhor [7].

Sobre a perda de espaço do Ruby

Eu tenho a impressão de que Ruby perdeu bastante espaço no imaginário de quem começa um projeto novo, principalmente fora do mundo Rails. Python virou a escolha quase automática para automações, scripts, análise de dados e IA. Parte disso vem da sintaxe acessível, mas acho que o motivo mais forte é um ciclo de adoção: quanto mais gente usa Python, mais bibliotecas, exemplos, tutoriais e profissionais aparecem; com esse material disponível, ainda mais pessoas e empresas escolhem Python.

Os dados ajudam a enxergar essa diferença, mas é importante não exagerar a conclusão. No Stack Overflow Developer Survey 2025, a adoção declarada de Python cresceu sete pontos percentuais em relação a 2024. O GitHub Octoverse 2025 também aponta forte crescimento de Python entre contribuintes e muita atividade em projetos de IA [12–13]. Isso não prova que o número absoluto de usuários de Ruby esteja diminuindo ano após ano, nem que Python seja melhor tecnicamente em tudo. Prefiro falar em menor protagonismo relativo. A associação histórica de Ruby com Rails também pode fazer muita gente ignorar as possibilidades da linguagem fora de aplicações web.

Depois de estudar esses oito exemplos, continuo achando Ruby uma linguagem muito interessante, mesmo sabendo que honestamente vou continuar utilizando Python, JavaScript ou outra stack num projeto diferente. Saber como Ruby lida com objetos, closures, lookup e metaprogramação serviu muito pra fins de conhecimento e saber que existem linguagens muito interessantes e complexas por aí, mesmo que atualmente não seja tão falado.

Repositório e testes

Deixei todos os oito exemplos de Ruby, os códigos de comparação com outras linguagens, a documentação e os testes disponíveis no GitHub: https://github.com/santana-iago/ruby-studycase. Também configurei o GitHub Actions para executar automaticamente os testes em Ruby, Python e JavaScript, além da comparação em Java. A última execução, em 21/10/2026, terminou sem erros. O código em C# ficou apenas como exemplo de comparação, sem teste automatizado. Já o

Ruby on Rails é abordado ao longo do relatório como referência para entender aplicações práticas dos recursos da linguagem, mas sem uma implementação própria no repositório.

Referências e fontes consultadas

[1] Akita, Fabio. Micro-Tutorial de Ruby — Parte I (2008). https://akitaonrails.com/2008/11/10/micro-tutorial-de-ruby-parte-i/

[2] Akita, Fabio. Micro-Tutorial de Ruby — Parte II (2008). https://akitaonrails.com/2008/11/10/micro-tutorial-de-ruby-parte-ii/

[3] Akita, Fabio. Micro-Tutorial de Ruby — Parte III (2008). https://akitaonrails.com/2008/11/10/micro-tutorial-de-ruby-parte-iii/

[4] Akita, Fabio. Anatomia de Ruby Blocks/Closures (2007). https://akitaonrails.com/2007/11/30/anatomia-de-ruby-blocks-closures/

[5] Katz, Yehuda. Metaprogramação em Ruby: é tudo sobre Self. Tradução publicada por Fabio Akita (2009). https://akitaonrails.com/2009/11/16/traducao-metaprogramacao-em-ruby-e-tudo-sobre-self/

[6] Akita, Fabio. Ruby Symbols (2007). https://akitaonrails.com/2007/11/26/ruby-symbols/

[7] Akita, Fabio. Existem Linguagens Melhores pra programar com LLMs? (11/10/2026). https://akitaonrails.com/2026/10/11/existem-linguagens-melhores-pra-programar-com-llms/

[8] AkitaOnRails, página principal. https://akitaonrails.com/

[9] Ruby. From Other Languages. https://www.ruby-lang.org/en/documentation/ruby-from-other-languages/

[10] Ruby. Official FAQ: Methods. https://www.ruby-lang.org/en/documentation/faq/7/

[11] Ruby Documentation. Refinements. https://docs.ruby-lang.org/en/master/syntax/refinements_rdoc.html

[12] Stack Overflow. Developer Survey 2025 — Technology. https://survey.stackoverflow.co/2025/technology

[13] GitHub. Octoverse 2025. https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/

[14] Ruby on Rails Guides. https://guides.rubyonrails.org/

[15] Ruby Documentation 3.3. Enumerator::Lazy. https://docs.ruby-lang.org/en/3.3/Enumerator/Lazy.html

[16] Ruby Documentation 3.3. Pattern matching. https://docs.ruby-lang.org/en/3.3/syntax/pattern_matching_rdoc.html